

The Alberta Buck - Demurrage and the Jubilee Fund

Perry Kundert

2026-04-30



Abstract

BUCKs accrue a flat 2%/yr linear demurrage on every BUCK in existence, computed at view time and never burned. The accrual splits by account flavour. Non-Carrying accounts (EOAs, user wallets) see `balanceOf` decline as locked fees grow. Carrying accounts (AMM pools, Notes, the Jubilee fund) keep `balanceOf` equal to their raw ERC-20 balance and carry the accrued fee out with every transfer; the recipient absorbs it via `_demurrage[to]`. The split is load-bearing: it is what lets stock Uniswap pools, treasuries, and other balance-tracking ERC-20 consumers co-exist with BUCKs across long idle periods. All demurrage ultimately settles into a Jubilee fund that mirrors the cumulative claim and burns against it at BuckCredit lien redemption.

Contents

1	Overview	4
2	Per-account state	4
3	Live fee calculation	5
4	Crystallisation	5
5	Transfers	6
5.1	Non-Carrying transfers (EOA sender)	6
5.2	Carrying transfers (Carrying-flagged sender)	6
5.3	Algebraic identity: total spendable is preserved	7
6	Mint and burn	7
6.1	Sequence on <code>Buck.mint(amount)</code>	7
6.2	Sequence on <code>Buck.burn(amount)</code>	7
6.3	Jubilee accrual	7
6.4	Why the Jubilee fund cannot be a pure view	8
7	Carrying accounts and the <code>isCarrying</code> flag	8
7.1	Why <code>balanceOf</code> equals <code>raw</code>	8
7.2	Setting the flag	9
7.3	Immutability after first approval	9
7.4	Why Carrying for service contracts	9
8	Worked example: one BUCK\$100 mint, no other events	9
9	Approve and consent	10
9.1	One approve covers both directions	10
9.2	Zero-allowance approves for receive-only consent	11
9.3	Approval freezes <code>isCarrying</code>	11
9.4	Non-Carrying senders	11
10	Lien redemption	11
10.1	Mechanics	11
10.2	Worked example: 50-year credit closed at year 50	12
10.3	Worked example: closed at year 25 (half-elapsed)	12
10.4	Why no compound interest is required	12
11	Conservation invariants	13
12	Implementation sketch	13
12.1	<code>Buck.sol</code> storage	13
12.2	<code>IdentityRegistry</code> storage additions	14
12.3	<code>Buck.sol</code> public/external view surface	14
12.4	ERC-20 hook dispatch	14
12.5	<code>IdentityRegistry</code> API additions	14

13 Open questions and policy choices	15
13.1 Should close clear _demurrage on the redeeming account?	15
13.2 Should the Jubilee fund itself be Carrying?	15
13.3 Long-idle Carrying accounts	15
13.4 Premium routing and the insurance pool	15
14 References	16

DRAFT

1 Overview

The BUCK has two account flavours, set at registration and frozen on first approval. Non-Carrying accounts – the default for EOAs – see `balanceOf` decay over time as demurrage locks fees inside the account. Carrying accounts – service contracts that hold BUCKs on behalf of others – keep `balanceOf` equal to raw and ride the accrued fee out with each transfer, where the recipient’s `_demurrage[to]` absorbs it. The Jubilee fund materialises at every mint/burn the cumulative claim $\text{BASE_RATE} * \text{area_under_supply}$, so `totalSupply` grows with the locked fees and unwinds at lien close. No fee is ever burned; demurrage is a redistribution, not a destruction.

The model has three load-bearing properties:

1. **Linear in time.** A BUCK held still earns demurrage at a linear rate on its raw balance. At BUCK\$100 raw and $\text{BASE_RATE} = 2\%/yr$, after one day the holder owes \$0.00547, after one year \$2, after fifty years \$100. No exponential decay; no compounding.
2. **No demurrage burn.** `totalSupply` is mutated only by user-initiated `mint()` / `burn()` and by Jubilee accrual that fires on those same calls. Transfers, including the Carrying flavour that delivers aged BUCKs to a recipient, never destroy supply.
3. **Conservation of effective spendable.** At any block, $\text{sum_a } \text{balanceOf}(\text{a}) + \text{sum_a } \text{balanceOfFees}(\text{a}) == \text{totalSupply}$ for Non-Carrying accounts; Carrying accounts contribute their full raw to `balanceOf` and their fee separately to `balanceOfFees`. No BUCK is created or destroyed by demurrage; locked fees in user accounts mirror the cumulative Jubilee claim and converge at lien redemption.

The two flavours dispatch automatically by sender:

- **Non-Carrying** (`isCarrying[from] == false`). `balanceOf` declines. Sender’s locked fees stay locked. Recipient receives fresh BUCKs (no inherited fee debt). Spendable cap on outflow: $\text{value} \leq \text{balanceOf}(\text{from})$.
- **Carrying** (`isCarrying[from] == true`). `balanceOf` equals raw. Sender’s basis untouched on outflow. Recipient’s `_demurrage[to]` absorbs the proportional carried fee. No spendable cap; the contract may transfer up to raw.

There is no separate `transferCarrying` function. Both `transfer` and `transferFrom` dispatch through the registry flag. Recipients pre-consent to receive carried fees from `spender` via the standard identity-bound `approve(spender, 0, ...)` call – the same CP-receipt mechanism that gates any identity-bound transfer.

2 Per-account state

Every account holds three storage values: `_balances[a]` (raw ERC-20 balance, inherited from OZ), `_demurrage[a]` (crystallised locked fees), and `_timestamp[a]` (block timestamp of the last balance-mutating event). Demurrage never modifies `_balances`; it grows `_demurrage` at crystallise-time and computes the live fee from $(_balances * \text{elapsed} * \text{RATE})$ at view time.

Field	Meaning
<code>_balances[a]</code>	Raw ERC-20 balance. Mutated only by user-initiated balance changes (mint, burn, transfer in, transfer out). Demurrage does not modify it.
<code>_demurrage[a]</code>	Cumulative locked fees accrued up through <code>_timestamp[a]</code> .
<code>_timestamp[a]</code>	Block timestamp of the most recent balance-mutating event for <code>a</code> . All fees through this instant are already folded into <code>_demurrage[a]</code> .

A freshly created account has `_balances = _demurrage = 0` and `_timestamp` unset. No fee accrues against zero raw, so the unset timestamp is irrelevant until raw becomes non-zero.

3 Live fee calculation

Three view functions expose demurrage state. `feeOwing` is the total fee accrued at the current block (always computed the same way). `balanceOfFees` and `balanceOf` split by account flavour: Non-Carrying treats fees as locked dust inside the account; Carrying treats fees as a separate carried-on-outflow obligation.

Let $\text{BASE_RATE_PER_SEC} = 2\%/yr / (365.25 \text{ days})$, and let `now` be `block.timestamp`.

```
function feeOwing(a):
    elapsed      = now - _timestamp[a]
    pending_fee  = _balances[a] * BASE_RATE_PER_SEC * elapsed
    return _demurrage[a] + pending_fee

function balanceOfFees(a):
    if identity.isCarrying(a):
        return feeOwing(a)                // carried-on-outflow
    return min(feeOwing(a), _balances[a])  // locked dust, capped at raw

function balanceOf(a):
    if identity.isCarrying(a):
        return _balances[a]                // raw, no decay
    fee = feeOwing(a)
    return fee >= _balances[a] ? 0 : _balances[a] - fee
```

The Carrying-balanceOf-equals-raw rule is what lets stock ERC-20 consumers (Uniswap pairs, treasuries, multi-sigs) co-exist with BUCKs over time. A pair’s cached reserve and its `balanceOf` remain in lock-step indefinitely; the K invariant is satisfiable across long idle periods without periodic `sync()`; LP-share computations from `balance.sub(reserve)` no longer underflow on a decayed balance. The fees aren’t lost – they ride out with the pair’s swap deliveries, and the swap recipient’s `_demurrage[to]` absorbs them.

For Non-Carrying accounts the math is the same as the prior model: `balanceOf` is what the holder can spend; the fee is locked dust inside the account. After 1 day at 2%/yr, BUCK\$100 reports `balanceOf` = \$99.99452..., after 1 year \$98, after 50 years \$0.

The view functions are $O(1)$ and require no on-chain state mutation – a wallet computes live demurrage at any moment without spending gas.

4 Crystallisation

Crystallisation folds the pending fee on `a` into `_demurrage[a]` and resets `_timestamp[a]` to `now`. It fires before every balance-mutating event for `a` so the account’s locked-fee bookkeeping is always current at the moment a transfer touches its raw.

```
function _crystallize(a):
    elapsed      = now - _timestamp[a]
    pending_fee  = _balances[a] * BASE_RATE_PER_SEC * elapsed
    _demurrage[a] = _demurrage[a] + pending_fee
    _timestamp[a] = now
```

The function is idempotent in time: two calls in the same block yield the same state as one. Pure read paths (`balanceOf`, `balanceOfFees`) never crystallise – they read storage and compute the view directly.

5 Transfers

Both `transfer` and `transferFrom` dispatch through `IdentityRegistry.isCarrying(from)`: a Non-Carrying sender crystallises and the recipient gets fresh BUCKs; a Carrying sender leaves its basis untouched and the recipient absorbs the carried fee. No separate `transferCarrying` function exists; the flavour is determined by who the sender is, not how they call.

```
function _update(from, to, value):
  if from == 0 || to == 0:
    return _mintBurnUpdate(from, to, value) // see "Mint and burn"

  if identity.isCarrying(from):
    _carryingTransfer(from, to, value)
  else:
    _nonCarryingTransfer(from, to, value)
```

A user wanting to receive aged BUCKs from a service contract approves that contract once via the existing identity-bound CP receipt; from then on every transfer that contract pushes to the user is Carrying.

5.1 Non-Carrying transfers (EOA sender)

The sender's residual continues to age from a fresh timestamp, the locked fees stay in the sender's account, and the recipient receives fresh BUCKs (no inherited fee debt). A spendable cap `value <= balanceOf(from)` prevents the sender from spending its locked dust.

```
function _nonCarryingTransfer(from, to, value):
  require(value <= balanceOf(from))
  _crystallize(from)
  _crystallize(to)
  _balances[from] -= value
  _balances[to] += value
  // _demurrage[from] retains the pre-existing locked fees.
  // _demurrage[to] is unchanged: recipient gets fresh BUCKs.
```

Both `_timestamp[*]` are set to `now` by crystallisation; the recipient's `_demurrage[to]` is unchanged so the new BUCKs land at age 0. The OZ super `_update` guards on raw, so the spendable check sits in front to prevent spending locked dust.

5.2 Carrying transfers (Carrying-flagged sender)

The sender's `_timestamp` and `_demurrage` are unchanged on outflow; the residual ages from the original basis. The recipient is crystallised first, then the carried fee `value * BASE_RATE_PER_SEC * (now - _timestamp[from])` is added to `_demurrage[to]`. The recipient's spendable rises by the net amount `value - carried_fee`, exactly the age-adjusted value of the incoming BUCKs.

```
function _carryingTransfer(from, to, value):
  require(value <= _balances[from])
  age_basis_seconds = now - _timestamp[from]
  carried_fee = value * BASE_RATE_PER_SEC * age_basis_seconds
  _crystallize(to)
  _balances[from] -= value
  _balances[to] += value
  _demurrage[to] += carried_fee
  // _timestamp[from] and _demurrage[from] are unchanged.
```

There is no spendable cap on the sender side: a Carrying contract may transfer up to its raw, with the carried fee obligation riding the BUCK to the recipient. The Carrying transfer is its own balance-mutating event for `to`, so the recipient is crystallised first; the carried fee is then added to the just-refreshed `_demurrage[to]`.

A Carrying transfer requires the recipient's prior consent via the identity-bound `approve` (see "Approve and consent"). This prevents a Carrying account from dumping unwanted fee debt on an unconsenting counterparty.

5.3 Algebraic identity: total spendable is preserved

For Non-Carrying accounts `balanceOf + balanceOfFees = _balances` holds at every block. For Carrying accounts the identity is broken on purpose: `balanceOf = _balances` and `balanceOfFees = feeOwing` are reported separately so that ERC-20 consumers see a stable balance even as the carried fee grows. Both transfer flavours preserve `sum_a _balances[a] = totalSupply` and the system fee debt `sum_a feeOwing(a) = BASE_RATE * area_under_supply`.

6 Mint and burn

Only `mint()` and `burn()` can mutate `totalSupply`. Both trigger Jubilee accrual before performing the user's intent: the Jubilee fund mints to itself the elapsed-period rate applied to the **current** total supply, bringing the materialised Jubilee balance up to the cumulative claim it has on the system.

6.1 Sequence on `Buck.mint(amount)`

```
function mint(amount):
    require(identity.isVerified(msg.sender))
    require(amount + _balances[msg.sender] <= storedLimit[msg.sender])
    _accrueJubilee() // see below
    _crystallize(msg.sender)
    premium = _computePremium(msg.sender, amount, storedLimit[msg.sender])
    super._mint(msg.sender, amount - premium)
    if premium > 0:
        _crystallize(insurancePool)
        super._mint(insurancePool, premium)
```

6.2 Sequence on `Buck.burn(amount)`

```
function burn(amount):
    require(amount <= balanceOf(msg.sender))
    _accrueJubilee()
    _crystallize(msg.sender)
    super._burn(msg.sender, amount)
```

6.3 Jubilee accrual

The Jubilee fund's raw balance grows **ONLY** when `_accrueJubilee()` runs, and `_accrueJubilee()` runs **ONLY** at the start of `mint()` or `burn()`. `totalSupply` grows by the same `delta`; the new BUCKs are backed by the locked fees that already exist in user accounts.

```
uint256 internal _jubileeLastUpdate; // initialised to genesis
```

```
function _accrueJubilee():
    elapsed = now - _jubileeLastUpdate
    if elapsed == 0:
```

```

    return
    delta = totalSupply() * BASE_RATE_PER_SEC * elapsed
    _jubileeLastUpdate = now           // set BEFORE super._mint to block re-entry
    _crystallize(address(this))
    super._update(0, address(this), delta) // direct, bypassing our _update override

```

The accrual mints fresh BUCKs to `address(this)` equal to the 2%/yr rate applied to the current total supply across the elapsed interval (the supply is piecewise constant between mint/burn events, so the integral is just `totalSupply * elapsed`). The new BUCKs themselves land at age 0; the crystallisation step folds Jubilee’s prior age into its `_demurrage[]` before the mint.

6.4 Why the Jubilee fund cannot be a pure view

A virtual `balanceOf(jubilee) = BASE_RATE * area_under_supply` view with no ERC-20 storage was considered and rejected. A `BuckCredit.close` must `super._burn` BUCKs from the Jubilee, which a virtual balance cannot satisfy without inventing a parallel supply concept. Real ERC-20 storage is operationally simpler, matches user intuition, and lets Carrying transfers from Jubilee push real raw to recipients.

7 Carrying accounts and the `isCarrying` flag

A Carrying account is any address whose `IdentityRegistry.isCarrying[a]` flag is true. Its `balanceOf` equals raw at all times; the demurrage it has accrued shows up in `balanceOfFees` and rides with every outflow. The flag exists primarily to make ERC-20 consumers (Uniswap, AMMs, treasuries) work correctly with BUCKs across long idle periods – a pair’s cached reserve and live `balanceOf` must agree, otherwise stock V2 swaps and `skim()` silently fail.

7.1 Why `balanceOf` equals raw

Stock ERC-20 consumers do not expect the balance under their custody to spontaneously decay. Uniswap V2’s `UniswapV2Pair` caches reserves on every event and reads `IERC20(token).balanceOf(this)` to detect deposits and withdrawals; `skim()` computes `balance.sub(reserve)` via `SafeMath`, `mint()` computes `balance.sub(reserve)` for new LP shares, and `swap()` checks `balanceAdjusted * balanceAdjusted > reserve * reserve` for the K invariant. Every one of these breaks (silently or with a revert) when `balanceOf` drifts below `reserve` due to demurrage.

By making `balanceOf(carrying) = _balances[carrying]`:

- `balance == reserve` holds across arbitrary idle periods.
- `skim()` sees a zero diff and returns cleanly.
- `mint()` attributes the new LP shares to the actual inflow.
- `swap()`’s K check is satisfiable; the fair-price math remains calibrated to the same constant the LPs deposited against.

The fees aren’t disappeared – they ride out with each Carrying transfer. When the pair pays a swap recipient, it calls `_carryingTransfer`: `_balances[pair]` drops by the gross value, `_balances[recipient]` rises by value, and `_demurrage[recipient]` absorbs `value * BASE_RATE_PER_SEC * (now - _timestamp[pair])`. The recipient’s spendable rises by the net (`value - carried_fee`); the swap-out delivers age-adjusted BUCKs even though the gross transfer was un-aged.

7.2 Setting the flag

The flag lives in `IdentityRegistry` and is set at registration time:

Path	Default	Override
<code>register()</code>	false (EOA)	not permitted – EOAs are always Non-Carrying.
<code>bindContract()</code>	true (service contract)	<code>bindContract(target, pk, E, isPublic, /isCarrying=false/)</code> for contract wallets that should crystallise like an EOA; multisigs operated by a single user, AA wallets, etc.

Carrying contracts are expected to be the typical case for service infrastructure (Notes pools, AMM pools, the Jubilee fund itself). Smart-contract wallets – multisigs, AA wallets representing one user’s volition – opt out at binding time so they crystallise like the EOA they replace.

7.3 Immutability after first approval

The flag is locked the first time any counterparty issues an identity-bound approve naming this address as the spender. Pre-approval changes are permitted (a freshly-deployed contract may flip its mind before going live); post-approval changes are not (recipients have already consented to a specific Carrying flavour, and the operator must not be able to retroactively change it).

```
// Buck.approve(spender, amount, E_spender, pi_CP):
// ... existing CP-receipt machinery ...
// identity.markApproved(spender) // freezes the spender's flag

function markApproved(address spender): // IdentityRegistry, external
    require(msg.sender == buck, "only Buck")
    if !carryingFrozen[spender]:
        carryingFrozen[spender] = true

function setIsCarrying(address target, bool value): // IdentityRegistry, external
    require(msg.sender == binderOf[target], "not binder")
    require(!carryingFrozen[target], "carrying frozen by approval")
    isCarrying[target] = value
```

A re-bound (re-deployed) contract resets through the standard `bindContract()` flow; in-place flag changes after first approval are simply not supported.

7.4 Why Carrying for service contracts

A service contract holds BUCKs temporarily on behalf of others. The Notes pool holds face value until a note is spent; an AMM pool holds liquidity waiting to be paired; the Jubilee fund holds collected demurrage waiting to be deployed. None of these accounts are economic principals – they are conduits, and the demurrage they "owe" is owed in substance by the parties whose value they hold for.

Forced to crystallise on outflow like an ordinary EOA, a Carrying contract would either degrade the funds it manages (absorbing the fee from its own holdings) or push fresh un-aged BUCKs onto recipients (mis-pricing the value). Carrying preserves the age basis: BUCKs that have sat in the Notes pool for six months arrives at the redeemer with six months of carried fee, and the redeemer’s `balanceOf` reflects it. The aged value is delivered to the eventual beneficiary; no party is short-changed.

8 Worked example: one BUCK\$100 mint, no other events

Setup at $t=0$:

```

mint(100) by alice (Non-Carrying EOA)
state:
  alice:    _balances=100, _demurrage=0,    _timestamp=0
  jubilee:  _balances=0,   _demurrage=0,    _timestamp=0
totalSupply: 100

```

After exactly 1 year (no other events):

```

state (storage unchanged):
  alice:    _balances=100, _demurrage=0,    _timestamp=0
  jubilee:  _balances=0,   _demurrage=0,    _timestamp=0
totalSupply: 100

views:
  feeOwing(alice)      = 100 * 2%/yr * 1yr = $2
  balanceOf(alice)     = $100 - $2 = $98      (Non-Carrying)
  balanceOfFees(alice) = $2                  (locked dust)

  feeOwing(jubilee)    = 0 * 2%/yr * 1yr = $0
  balanceOf(jubilee)   = $0

```

The "missing" BUCK\$2 is locked in alice's account and will be swept into the Jubilee fund at the next mint/burn event. At $t=1\text{yr}$ a second account does `mint(1)`:

```

1) _accrueJubilee():
   elapsed = 1yr, totalSupply = $100, delta = $2
   _crystallize(jubilee): no-op (raw=0)
   super._update(0, jubilee, $2): _balances += $2, totalSupply = $102
2) _crystallize(alice):
   _demurrage += $2, _timestamp = t1
3) super._mint(bob, $1):
   bob._balances = $1, totalSupply = $103

state after:
  alice:  _balances=100, _demurrage=$2, _timestamp=t1
  bob:    _balances=$1,  _demurrage=0,  _timestamp=t1
  jubilee: _balances=$2, _demurrage=0,  _timestamp=t1
totalSupply: $103

```

If Jubilee is registered as a Carrying account then `balanceOf(jubilee) = $2` holds at every later block, with its accrued self-demurrage visible in `balanceOfFees`. If Jubilee is left as Non-Carrying (the current default – `address(this)` is not auto-bound at deploy time), then a year later `feeOwing(jubilee) = $0.04` and `balanceOf(jubilee) = $1.96`. Either way alice's `balanceOf` continues to decline and the system fee debt $\text{sum}_a \text{feeOwing}(a) = \text{BASE_RATE} * \text{area_under_supply}$ remains exact.

9 Approve and consent

The identity-bound `Buck.approve(spender, amount, E_spender, pi_CP)` serves a dual role. Its primary job is the existing CP-receipt: it produces the identity binding that gates identity-bound transfers between `sender` and `spender`. Its second job is the recipient's consent to receive Carrying transfers from `spender`, plus the freeze of `isCarrying[spender]` against future operator change.

9.1 One approve covers both directions

The CP receipt is checked symmetrically for any transfer between the two identities, regardless of which one ends up as `msg.sender`. When the spender is a service contract (`isCarrying = true`) and pushes BUCKs to the recipient, the recipient's prior approve has already established the receipt; the contract's Carrying transfer proceeds through the same identity-bilateral check as any other identity-bound transfer.

9.2 Zero-allowance approves for receive-only consent

When a recipient simply wants to enable an incoming Carrying transfer (e.g., to receive future deployments from the Jubilee fund) without granting any spend allowance, they call `Buck.approve(carryingAccount, 0, E_carryingAccount, pi_CP)`. This creates the receipt fragment without granting allowance. `amount=0` is exactly the semantic the user wants: identity binding plus consent-to-receive without spending authorisation.

9.3 Approval freezes `isCarrying`

The first approve naming `carryingAccount` as the spender freezes `isCarrying[carryingAccount]`. A wallet UI presenting an approve confirmation should surface the current `isCarrying` value so the user can see whether they're consenting to a Carrying-flavour relationship; once they approve, the operator can no longer change the flavour.

9.4 Non-Carrying senders

The existing identity-bound approve applies unchanged: a Non-Carrying sender approves a spender with a non-zero allowance and a CP receipt; `transferFrom` checks both the allowance and the receipt. Non-Carrying transfers carry no fee debt to the recipient, so no extra consent dimension is involved.

10 Lien redemption

A BuckCredit NFT backs a BUCK mint via an off-chain insured asset; when its "close" mechanic is invoked the lien is released and BUCKs equal to the originally-insured face value are burned. The burn is sourced jointly from the original holder and the Jubilee fund in proportion to elapsed lifetime: at $t=L$ (full term) the entire face comes from Jubilee's accumulated balance; at $t=0$ it would all come from the holder; partial closures split linearly.

10.1 Mechanics

For a BuckCredit insured at face F with lifetime L , closed at elapsed e (with $e \leq L$):

```
function close(BuckCreditId):
  F = creditFace(id)
  L = creditLifetime(id)
  e = now - creditMintTime(id)

  fromJubilee = F * min(e, L) / L
  fromAlice   = F - fromJubilee

  Buck._accrueJubilee()           // bring Jubilee current
  Buck._crystallize(alice)
  Buck._crystallize(jubilee)

  require(_balances[alice] >= fromAlice, "alice underfunded")
  require(_balances[jubilee] >= fromJubilee, "jubilee underfunded")

  Buck.super._burn(alice, fromAlice)
  Buck.super._burn(jubilee, fromJubilee)
  // BuckCredit then releases the asset to alice and self-destructs.
```

10.2 Worked example: 50-year credit closed at year 50

Alice mints BUCK\$100 at $t=0$ ($F=100$, $L=50\text{yr}$, no premium for simplicity). No further events for 50 years. At $t=50\text{yr}$, BuckCredit calls `close`:

```
1) _accrueJubilee():
    delta = 100 * 2%/yr * 50yr = $100
    super._update(0, jubilee, $100): _balances[jubilee] = $100
    totalSupply: $100 -> $200

2) _crystallize(alice):
    _demurrage[alice] = $100, _timestamp[alice] = t50

3) Split:
    fromJubilee = $100 * 50/50 = $100
    fromAlice   = $100 - $100 = $0

4) Burns:
    super._burn(alice, $0): no change
    super._burn(jubilee, $100): _balances[jubilee] = 0
    totalSupply: $200 -> $100

5) Asset returned to alice; BuckCredit self-destructs.
```

Alice's `_balances` remains \$100, but `_demurrage[alice] = $100`, so `balanceOf(alice) = $0`. The original mint has been fully redeemed: alice paid the \$100 of demurrage over 50 years (it accrued in `_demurrage[alice]` at lien-close time), and the Jubilee fund burned the matching \$100 it had accumulated.

10.3 Worked example: closed at year 25 (half-elapsed)

Same setup, close at $t=25\text{yr}$:

```
1) _accrueJubilee():
    delta = 100 * 2%/yr * 25yr = $50
    _balances[jubilee] = $50, totalSupply: $100 -> $150

2) _crystallize(alice):
    _demurrage[alice] = $50, _timestamp[alice] = t25

3) Split: fromJubilee = $50, fromAlice = $50

4) Burns:
    super._burn(alice, $50): _balances[alice] = $50
    super._burn(jubilee, $50): _balances[jubilee] = 0
    totalSupply: $150 -> $50

5) Asset returned, BuckCredit destroyed.
```

After early closure alice retains \$50 of raw with `_demurrage[alice] = $50`, giving `balanceOf(alice) = $0` immediately. The implementation may choose to clear `_demurrage[alice]` as part of `close` (freeing the residual immediately) or to leave it (let it re-amortise over the remaining "credit-free" period); this is a BuckCredit policy choice, not a Buck demurrage primitive.

10.4 Why no compound interest is required

$\text{BASE_RATE} * \text{totalSupply} * \text{elapsed}$ is exactly the amount the Jubilee can offer at lien close, and it equals exactly what the holder has accrued in locked fees over that same elapsed period. Both sides sum to F over L for a credit that ages its full lifetime. No account compounds interest on demurrage; the rate is applied to raw `_balances` linearly, and Jubilee's accumulation tracks the same linear integral across all outstanding BUCKs.

In practice the Jubilee deploys its idle holdings to revenue-generating subsidiaries (AMM pools, parametric insurance, etc.) which earn at least the owed self-demurrage and return profits to the

pool, so the worked-example simplification of "Jubilee accrues self-demurrage like any other account" rarely matters in production.

11 Conservation invariants

These are verifiable on every block, after every operation.

1. **Raw conservation.** `sum_a _balances[a] = totalSupply`. ERC-20 super-state. `mint`, `burn`, and `_accrueJubilee` are the only writes that move `totalSupply`.
2. **Spendable identity (Non-Carrying).** For every Non-Carrying `a`: `balanceOf(a) + balanceOfFees(a) = _balances[a]`. Direct from the cap `balanceOfFees <= _balances` and the definition `balanceOf = _balances - balanceOfFees`.
3. **Spendable identity (Carrying).** For every Carrying `a`: `balanceOf(a) = _balances[a]` and `balanceOfFees(a)` reports the carried-on-outflow obligation separately. The sum `balanceOf + balanceOfFees > _balances` is intentional; the carried fee is not subtracted from the reported balance.
4. **Demurrage rate identity.** Between any two adjacent mint/burn events at times `t_0` and `t_1` (`totalSupply` constant): $d(\text{sum_a feeOwing}(a)) / dt = \text{BASE_RATE} * \text{totalSupply}$. The system fee debt grows at exactly the demurrage rate times the supply. Across multiple intervals this integrates to `BASE_RATE * area_under_supply`, which is the cumulative Jubilee target. When `_accrueJubilee` materialises a delta, it shifts that growth from "implicit in user accounts" to "live in `_balances[address(this)]`".
5. **Carrying-transfer invariant.** A transfer whose sender has `isCarrying = true` preserves `sum_a feeOwing(a)` (apart from rounding). The carried amount moves from the sender's implicit basis to the recipient's explicit `_demurrage[to]`.
6. **Non-Carrying transfer invariant.** A transfer whose sender has `isCarrying = false` crystallises `balanceOfFees(from)` into `_demurrage[from]` and leaves `sum_a balanceOf(a)` unchanged; the recipient's incoming BUCKs land at age 0, so the system fee debt grows only by the elapsed-time integral.
7. **isCarrying-immutability invariant.** For any `a` such that `carryingFrozen[a] = true`, `isCarrying[a]` cannot be modified by any subsequent transaction. `carryingFrozen[a]` becomes true the first time `Buck.approve` names `a` as the spender (via the registry's `markApproved(a)` hook).

12 Implementation sketch

12.1 Buck.sol storage

```
mapping(address => uint256) internal _demurrage;
mapping(address => uint64)   internal _timestamp;
uint64                      internal _jubileeLastUpdate;
```

Per-account `_balances[a]` continues to be inherited from OZ ERC20. No prior cumulative-index mapping; the `(_demurrage[a], _timestamp[a])` pair captures all per-account state, and Jubilee accrual reads `totalSupply()` and the elapsed time directly.

12.2 IdentityRegistry storage additions

```
mapping(address => bool)    public isCarrying;
mapping(address => bool)    public carryingFrozen;
mapping(address => address) public binderOf;
address                    public buck;           // set once via setBuck()
```

`isCarrying[a]` is set at registration / binding time and read on every `Buck._update` to dispatch the transfer flavour, plus on every `balanceOf` / `balanceOfFees` view to select the reporting branch. `carryingFrozen[a]` locks `isCarrying[a]` once any counterparty has approved. `binderOf[a]` records the `msg.sender` of `bindContract` (zero for EOAs). `buck` is a one-time governance setter naming the authorised Buck contract – the only address permitted to call `markApproved`.

12.3 Buck.sol public/external view surface

```
function feeOwing(address a)          external view returns (uint256);
function balanceOfFees(address a)     external view returns (uint256);
function balanceOf(address a)         public view override returns (uint256);
function rawBalanceOf(address a)      external view returns (uint256);
function jubileeBalance()             external view returns (uint256);
```

There is no `transferCarrying` public function. The two transfer flavours are dispatched internally based on `IdentityRegistry.isCarrying(msg.sender)` (or from for `transferFrom`).

12.4 ERC-20 hook dispatch

```
function _update(address from, address to, uint256 value) internal override {
    if (from == address(0) || to == address(0)) {
        // mint / burn
        _accrueJubilee();
        if (from != address(0)) {
            require(value <= balanceOf(from), "BUCK: amount exceeds spendable");
            _crystallize(from);
        }
        if (to != address(0)) _crystallize(to);
        super._update(from, to, value);
        return;
    }

    if (identity.isCarrying(from)) {
        _carryingTransfer(from, to, value);
    } else {
        _nonCarryingTransfer(from, to, value);
    }
}
```

12.5 IdentityRegistry API additions

```
// One-time governance setter for the authorised Buck contract.
function setBuck(address _buck) external;

// register() leaves isCarrying[msg.sender] = false; binderOf is unset.
// bindContract() takes an explicit isCarrying_ flag and records msg.sender
// in binderOf[target].
function bindContract(
    address target,
    BN254.G1Point calldata pk,
    ElGamalCT calldata E,
    bool isPublicIdentity_,
    bool isCarrying_
) external;
```

```
// Pre-approval reconfiguration. Reverts if carryingFrozen[target].
function setIsCarrying(address target, bool value) external;

// Called from Buck.approve() to freeze the spender's flag.
// Reverts unless msg.sender == buck. Idempotent.
function markApproved(address spender) external;
```

Receipt fragments and identity binding (existing `IdentityRegistry` flow) are layered on top unchanged: the identity check for both flavours of transfer happens at the public function boundary, before the demurrage logic runs. `Buck.approve` calls `identity.markApproved(spender)` immediately after writing the CP receipt so the spender's `isCarrying` flag becomes immutable from that moment on.

13 Open questions and policy choices

13.1 Should close clear `_demurrage` on the redeeming account?

After early closure ($e < L$) the holder's `_demurrage[a]` reflects the locked fees from before the close. The holder retains some raw, but `balanceOf(a)` reads as zero until those fees re-amortise. `close` may zero `_demurrage[a]` (freeing the residual immediately) or leave it; this is a BuckCredit policy choice, not a Buck demurrage primitive.

13.2 Should the Jubilee fund itself be Carrying?

`address(this)` is not auto-bound at Buck deploy time – it cannot be, since `target.code.length > 0` fails inside the constructor. Governance can call `bindContract(buck, ..., isCarrying=true)` post-deploy to register Jubilee as a Carrying account, after which `balanceOf(jubilee)` equals raw and Jubilee outflows ride the carried fee. Until that call happens, Jubilee is treated as Non-Carrying: its `balanceOf` declines as its self-demurrage accumulates. In production the Jubilee deploys idle holdings to revenue-generating subsidiaries that offset the self-demurrage, so the distinction matters less than it appears.

13.3 Long-idle Carrying accounts

A Carrying account whose policy is to hold BUCKs indefinitely will accumulate self-demurrage; a transfer of the full raw will push a large carried fee onto the recipient. The contract enforces only `value <= _balances[from]`; the recipient's prior identity-bound approve is the social control. Wallet UIs should surface the carried-fee amount during the approve flow so the recipient sees what they're consenting to.

13.4 Premium routing and the insurance pool

A mint's premium deposit is routed to `insurancePool`, today a single stand-in address that every test and simulation binds Carrying through the registry, as a contract working on its members' behalf: its `balanceOf` stays equal to its raw balance, and a refund on `burn` carries the refunded share of its accrued age back to the member. In the intended design each credit's deposit goes to that credit's insurer, whose premium pool is Carrying in just the same way. See *The Insurance Pool: an Interim Stand-in* in alberta-buck-ethereum.org.

14 References

- `src/Buck.sol` – the implementation following this design.
- `src/IdentityRegistry.sol` – identity binding, the `isCarrying` flag, `markApproved` hook, and approve receipts.
- `test/UniswapV2Integration.t.sol` – end-to-end validation that real Uniswap V2 pairs/routers co-exist with BUCKs across long idle periods under the Carrying-balanceOf- equals-raw rule.
- `alberta-buck-ethereum.org` – the umbrella design document introducing BUCK_K and the BuckCredit insurance machinery.
- `alberta-buck-identity-example.org` – worked cryptographic examples for the identity-bound approve flow.

DRAFT